LABORATORIUM 6A. PORT SZEREGOWY USART

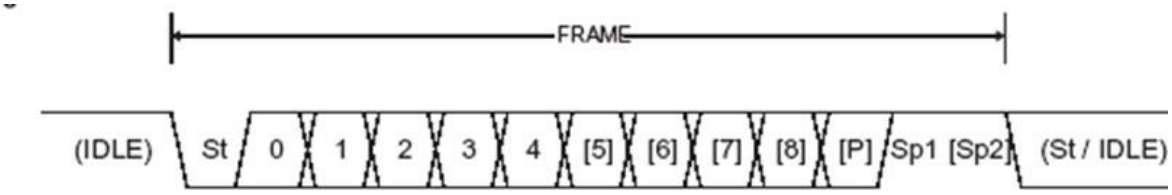
Cel laboratorium:

Celem laboratorium jest nabycie praktycznych umiejętności z programowania mikrokontrolerów z wykorzystaniem portu szeregowego USART.

Zakres tematyczny zajęć:

1. Budowa portu USART

Mikrokontroler AVR ATmega posiada elastyczny port szeregowy USART, który może być używany do komunikacji szeregowej z innymi urządzeniami, takimi jak komputery, szeregowe moduły GSM, GPS itp. Podczas wysyłania / odbierania danych niektóre bity są dodawane w celu poznania początku / końca danych itp. Powszechnie używana struktura to: 8 bitów danych, 1 bit początkowy (logika 0) i 1 bit stopu (logika 1), jak pokazano na rys. 6a.1:



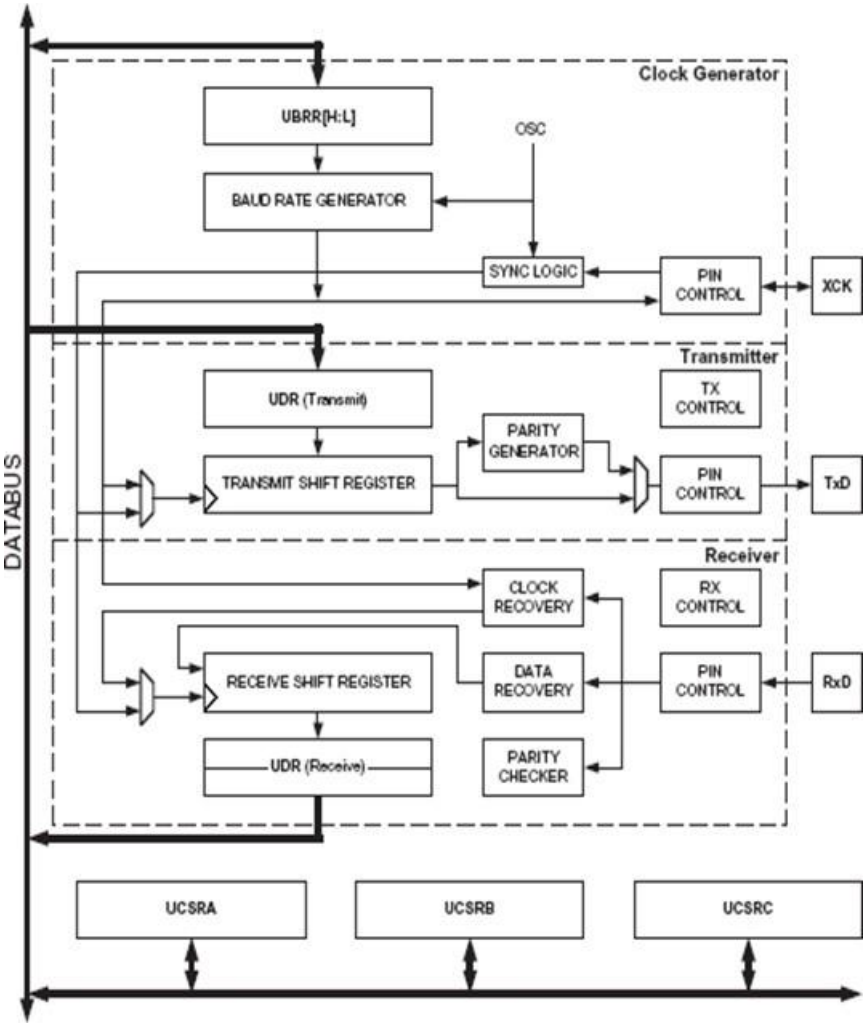
St	Start bit, always low.
(n)	Data bits (0 to 8).
P	Parity bit. Can be odd or even.
Sp	Stop bit, always high.
IDLE	No transfers on the communication line (RxD or TxD). An IDLE line must be high.

Rys. 6a.1. Struktura ramki portu USART

Istnieją również inne obsługiwane formaty ramek dostępne w UART, takie jak bit parzystości, zmienne bity danych (5-9 bitów danych).

Szybkość transmisji to "Liczba bitów na sekundę (bps)", znana również jako szybkość transmisji w systemie binarnym.. Istnieją pewne standardowe szybkości transmisji, np. 1200, 2400, 4800, 19200, 115200 bps itp. Zwykle 9600 bps jest używane tam, gdzie prędkość nie jest kwestią krytyczną.

Mikrokontroler Atmega32 posiada tzw. Universal Asynchronous Receiver Transmitter (UART), który może być używany do komunikacji z dowolnym innym urządzeniem RS-232, takim jak PC.
Uproszczony schemat blokowy portu przedstawiono na rys. 6a.2



Rys. 6a.2. Schemat blokowy portu USART

Do najważniejszych rejestrów specjalnych portu USART w AVR zaliczamy:

- **Rejestry: nadawczy i odbiorczy portu USART- UDR**

Bit	7	6	5	4	3	2	1	0	
	RXB[7:0]								UDR (Read)
	TXB[7:0]								UDR (Write)
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

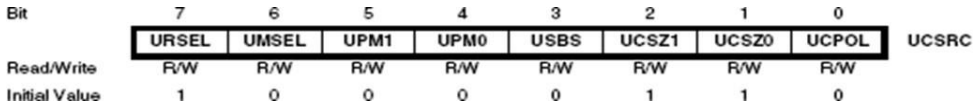
- **Rejestr kontrolno-sterujący portu USART: A- UCSRA**

Bit	7	6	5	4	3	2	1	0	
	RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM	UCSRA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

- **Rejestr kontrolno-sterujący portu USART: B- UCSRB**

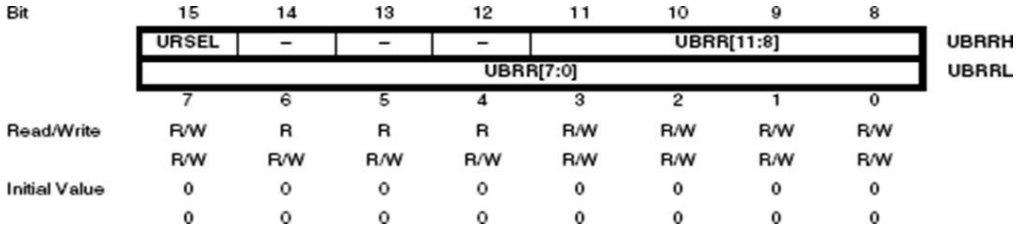
Bit	7	6	5	4	3	2	1	0	
	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	UCSRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Rejestr kontrolno-sterujący portu USART: C- UCSRC**



Bit	7	6	5	4	3	2	1	0	
	URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL	UCSRC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	0	0	0	0	1	1	0	

- **UCPOL** - tylko w trybie synchronicznym określa polaryzację sygnału zegarowego



Bit	15	14	13	12	11	10	9	8	
	URSEL	–	–	–	UBRR[11:8]				UBRRH
	UBRR[7:0]								UBRRL
	7	6	5	4	3	2	1	0	
Read/Write	R/W	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

- **URSEL** - bit określający dostęp do starszej lub młodszej części rejestru. Przy zapisie do UBRRH powinien być ustawiony w stan niski

Oznaczenia poszczególnych flag dostępne jest w materiałach wykładowych:

IIS5.3 Systemy wbudowane Wykład3.pdf

Układy peryferyjne mikrokontrolera AVR – porty, timery, system przerwań, zarządzanie energią, układy transmisji szeregowej

Do obsługi UART najpierw należy określić transmisję w UART. Po inicjalizacji UART może wykonywać zadania odczytu i zapisu. W tym przypadku rejestry bufora danych odczytu i zapisu mają ten sam adres wejścia-wyjścia (I/O), zwany UDR (USART Data Request). Rejestr bufora danych transmisji (TXB) będzie determinacją dla danych zapisanych w lokalizacji rejestru UDR. Odczytanie lokalizacji rejestru UDR spowoduje zwrócenie zawartości rejestru bufora odbieranych danych (RXB).

Inicjalizacja sterownika portu USART

```
void usart_init(void)                    //funkcja inicjalizująca usart
{
    UBRRH = (BAUD_PRESCALE >> 8);        //wpisanie starszego bajtu
    UBRRL = BAUD_PRESCALE;               //wpisanie mlodszego bajtu

    //UCSRA bez zmian - 0x00
    UCSRB = (1<<RXCIE)|(1<<RXEN)|(1<<TXEN);    //aktywne przerwanie od
odbioru oraz zmiana trybu działania pinów D0 i D1

    UCSRC = (1 << URSEL) | (1 << UCSZ0) | (1 << UCSZ1);     //praca
asynchroniczna, brak kontroli parzystości, 1 bit stopu, 8 bitów danych

}
```

Odbiór danych można dokonać dwoma sposobami:

- programowo poprzez monitorowanie bitu RXC w UCSRA. Bit RXC wskazuje stan odebrania danej.

```
unsigned char UART_RxChar()
{

        while ((UCSRA & (1 << RXC)) == 0);/* Wait till data is received */
        return(UDR);            /* Return the byte */
}
```

- lub poprzez przerwanie:

```
ISR(USART_RXC_vect)         //przerwanie od odbioru danej
{
    static char a;          //zmienna pomocnicza
    a = UDR;                //zapis odebranej danej
    a ^= 0xFF;              //operacja bitowa XOR
    UDR = a;                //wysłanie danej zwrotnej
}
```

Nadawanie poprzez port można dokonać:
- monitorując bit UDRE z UCSRA. Gdy flaga UDRE jest ustawiona, oznacza że bufor nadawczy jest pusty i gotowy do przyjęcia kolejnego bajtu:

```
void UART_TxChar(char ch)
void UART_TxChar(char znak)
{

        while (! (UCSRA & (1<<UDRE)));  /* Wait for empty transmit buffer */
        UDR = znak ;
}
```

- lub w przerwaniu od timera, który służy jako znacznik czasu wysyłania danych poprzez port:

```
void timer0_init(void)
{
    //praca w przerwaniu od przepelnienia, preskaler 256, wysylanie danych co 3,5ms
    TCCR0 |= (1<<CS02);
    TIMSK |= (1<<TOIE0);
}

 ISR(TIMER0_OVF_vect)
{
    UDR = PINB;           //Port B jako źródło danych
}
```

Pytania kontrolne:

Pytanie 1. Jak należy skonfigurować port USART do trybu nadawanie z prędkością 9600 bodów (bps) dla F_CPU= 4 MHz.
Pytanie 2. W jaki sposób określić moment wysłania kolejnej danej poprzez port USART.

Zadanie 1:
Napisać program, który spowoduje wysłanie ciągu znaków "**Imię i Nazwisko**" (indywidualnie studenta) na terminal z wykorzystaniem USART. W tym celu należy odpowiednio skonfigurować ustawienia transmisji szeregowej:
(baud rate = 9600, ramka: 8 bitów danych, brak kontroli parzystości, 1 bit stopu) oraz, w oparciu o funkcję przesyłania znaku, napisać funkcję wysyłającą ciąg znaków.

```
#define F_CPU 800000UL // ustawienie częstotliwości zegara – proponowane 8MHz
#define BAUDRATE 9600 //określenie prędkości przesyłania
#define BAUD_PRESCALLER (((F_CPU / (BAUDRATE * 16UL))) - 1) //wyznaczenie zawartości rejestru UBRR ze wzoru

void USART_Init(void)
{
/* ustawienie baud */
UBRRH = (uint8_t)(BAUD_PRESCALLER>>8);
UBRRL = (uint8_t)(BAUD_PRESCALLER);
//konfiguracja… (baud rate = 9600, ramka: 8 bitów danych, brak kontroli parzystości, 1 bit stopu)
}

void USART_putchar(unsigned char data)
{
while (!(UCSRA & (1<<UDRE)));
UDR = data;
}
```

Zadanie 2:
Do programu z zadania 1 dodać funkcję odbioru danych. Na terminal należy wysłać wiadomość: "Nacisnij klawisz [x]" (student wybiera indywidualnie znak). Następnie program powinien oczekiwać na wprowadzenie wybranego znaku z poziomu klawiatury komputera PC, a po jego podaniu nadać wiadomość potwierdzającą odbiór (np.. “znak prawidlowy”).
W przypadku podania złego znaku (litery) należy wygenerować kod błędu (np. "błędny znak").
Tip:

```
unsigned char UART_RxChar()
{
while ((UCSRA & (1<<RXC))==0); /* Wait till data is received */
return UDR;/* Return the byte */
```

}

Zadanie 3:

Należy podłączyć diody przewodami łączeniowymi według schematu:
LED_0 → (PA0)
LED_1 → (PA1)
LED_2 → (PA2)
LED_3 → (PA3)
LED_4 → (PA4)
LED_5 → (PA5)
LED_6 → (PA6)
LED_7 → (PA7)
W układzie jak powyżej (baud rate = 9600, ramka: 8 bitów danych, brak kontroli parzystości, 1 bit stopu) napisać program, który:

a) będzie powodował cykliczne miganie diodą LED_0 z wykorzystaniem przerwania od porównania Timera 0,

b) umożliwi sterowanie pozostałymi diodami z klawiatury komputera PC ([1] – LED_1, [2] – LED_2, itd.) z wykorzystaniem transmisji szeregowej. Po wciśnięciu klawisza [x], jego wartość powinna zostać wyświetlona na terminalu komputera PC, a wybrana dioda powinna zostać przełączona w stan przeciwny. Dodatkowo przyciski [c] i [s] powinny odpowiednio wyłączać (clear) i włączać (set) wszystkie diody od LED_1 do LED_7. Wciśnięcie jakiegokolwiek innego klawisza powinno zwrócić wiadomość o błędzie. Do realizacji odbioru danych można w tym zadaniu wykorzystać obsługę przerwania od zakończenia odbioru przez USART (USART_RXC_vect).

Tip:
Załączenie obsługi przerwań od odbioru danych następuje w rejestrze UCSRB.